Agentic support for a catalog that will not fit in a prompt

Three hundred SKUs, three hundred service manuals, and a three-billion-parameter model on a CPU. This is where the line between them sits.

Deterministic — Python and SQL **Language model** — four call sites

Every marker below is one or the other.

01 What it is for

FitForge sells home fitness equipment direct to consumers. Every model carries its own service manual, parts catalog and warranty terms. Customers arrive in chat with a broken machine and expect support to know which machine that is.

The system has five jobs.

1. **Identify the exact model first.** Every fact downstream — troubleshooting steps, part numbers, warranty terms — is keyed on it.
2. **Troubleshoot iteratively.** One check at a time, interpret the answer, continue until the fault is fixed or provably unfixable.
3. **Handle the parts path.** Check warranty, take payment when the part is not covered, ship to the address on file.
4. **Track unrelated problems at once.** Customers raise a treadmill fault and a bike fault in the same breath.
5. **Hand off with nothing lost.** When it cannot resolve an issue, a human inherits the full context.

It runs entirely on open-source software, self-hosted, with no external API calls. The reference deployment has no usable GPU. That constraint is not incidental — it is what forced the architecture to be honest about which decisions belong to a language model and which do not.

02 The governing principle

> **The model decides what to say and what to ask next.** *It never decides what is true, what is covered, what something costs, or what gets charged.*

Warranty verdicts, prices, part numbers, safety refusals and escalation triggers are ordinary Python and SQL. The `policy/` package does not import the model client — the boundary is enforced by the absence of the import, not by convention.

03 Structure

Two directories carry most of the meaning. `policy/` holds every decision that costs money or carries liability, and is pure Python. `agent/` holds the model, and is fenced.

```
web/                        one Vite app, two documents
  src/chat/                   customer widget                    →  /
  src/console/                human-agent console                →  /console

services/api/app/
  main.py                     FastAPI — chat WS, console WS, REST
  agent/
    graph.py                  the state machine; every edge a named condition
    nodes.py                  precheck · route · identify · open_issue · diagnose
    commerce_nodes.py         select_part · quote · confirm · pay · escalate
    prompts.py                four prompts and the schemas that bound them
    llm.py                    the only module that talks to Ollama
  policy/                     warranty · safety · escalation   ← no model, ever
  tools/                      identity · knowledge · catalog · commerce · manuals

services/ingest/            extract · ocr · chunk · embed · pipeline
services/mockpsp/           stand-in payment provider

db/migrations/              21 tables — the design is legible from the schema
seed/                       generates the 300-SKU catalog and its manuals
evals/ tests/ web/e2e/      golden sessions · unit coverage · browser suites
```

04 What is used for what

One line per job. Everything here is open source and self-hosted; nothing in this list bills anyone.

THE JOB	WHAT DOES IT	WHY THIS ONE
Relational database catalog, customers, orders, parts, warranty terms, issue threads, audit log	`PostgreSQL 17`	One system for every kind of state the agent has.
Vector database semantic search over manual chunks	`pgvector` inside that same Postgres	3,100 chunks is nothing for pgvector. A separate vector store adds a second system to run and back up, to solve a scaling problem that does not exist — and costs transactional consistency with the catalog.
Keyword search exact model numbers, part codes	`Postgres tsvector`	Vectors are bad at exact tokens. `FF-TT-PACER-550` has to match literally.
Search fusion	`Reciprocal Rank Fusion` in SQL	No cross-encoder reranker — the worst latency-to-benefit ratio in the stack on CPU, over an already model-scoped candidate set.
Agent state machine routing, the gate, the diagnostic loop	`LangGraph 0.2.60`	Explicit graph with named edges. The model stays inside nodes; control flow stays in code.
Durable sessions surviving a restart mid-conversation	`langgraph-checkpoint-postgres`	Checkpoints land in the same Postgres. No extra store.
Language model diagnostic steps, phrasing	`qwen2.5:3b-instruct`	Best JSON adherence per token at a size a CPU can serve.
Router model intent classification only	`qwen2.5:1.5b-instruct`	Classification needs no depth, and this is roughly 10x faster.
Embeddings	`nomic-embed-text` 768-dim	Strong retrieval, served over HTTP — so no torch in the application image.
Model serving	`Ollama`	Free, local, offline, and OpenAI-compatible.

THE JOB	WHAT DOES IT	WHY THIS ONE
Model client	`openai 1.59.6`	Speaks the shape Ollama, vLLM, llama.cpp, TGI and LM Studio all implement. Moving to a GPU box is three lines of `.env`.
Structured output forcing valid JSON from a 3B model	`response_format: json_schema`	Constrains decoding itself. Ollama silently ignores its native `format` field on the OpenAI route.
PDF text extraction	`pypdfium2 5.13`	BSD-3/Apache-2.0. Deliberately **not** PyMuPDF, which is AGPL and a licensing hazard for a commercial support product.
OCR scanned and photographed manuals	`OCRmyPDF 17 + Tesseract`	Deskew, clean and rotate for free; writes a real text layer back into the PDF.
Manual generation synthetic corpus and sample manuals	`ReportLab 4.2`	Vector line art, tables and panels, born-digital output.
HTTP API and WebSockets	`FastAPI + Uvicorn`	WebSockets built in; typed tool contracts fall out of the models.
Validation every tool boundary	`Pydantic v2`	A hallucinated part number becomes a tool error, not a sentence.
Pub/sub pushing escalations to the console	`Redis 7`	—
Payments	`Mock PSP` in this repo	The real flow — tokens, idempotency keys, declines — with zero cost and zero PCI surface.
Frontend	`React 18 + Vite 6`	One dev server hosts both UIs as separate documents and proxies the API, so the browser talks to one origin.
Browser tests	`Playwright`	Drives both UIs for real. Found two bugs the unit tests could not.
Unit tests	`pytest`	56 tests over the deterministic paths.
Tracing (optional)	`Langfuse, self-hosted`	Full LLM tracing without a SaaS bill. Off by default; every call is still recorded to `llm_calls`.
Runtime	`Docker Compose`	`python:3.12-slim`, `node:22-alpine`.

What is deliberately absent

A dedicated vector database

Qdrant, Weaviate, Milvus — the corpus is far too small to justify a second system to run, back up and keep consistent with the catalog.

A cross-encoder reranker

The worst latency-to-benefit ratio in the stack on CPU, over a candidate set that is already scoped to one machine.

PyMuPDF

Nicer API, better table extraction, AGPL-3.0. A cheap decision now and an expensive one to reverse after legal review.

LangChain retrievers and chains

The retrieval is forty lines of SQL that has to stay auditable.

Any hosted API

Model, embedding, OCR, payment. The constraint on this build is no paid services.

Fine-tuning

Retrieval quality and prompt structure carry more weight for the effort.

05 How it works

There are two timelines, and they are completely separate. Conflating them is the usual misreading: there is no separate body of knowledge the model answers from. The database *is* the documents.

Timeline A — a PDF becomes rows

Runs once per manual, offline. No language model is involved at any step, which is why every marker here is steel.

Ingest pipeline	ONCE PER MANUAL · NO MODEL CALLS
`classify`	Text-density heuristic per page: **born-digital or scanned?**
`ocr`	Scanned pages through ocrmypdf and Tesseract — deskew, clean, rebuild a text layer.
`extract`	pypdfium2. Permissively licensed, deliberately not AGPL PyMuPDF.
`score`	OCR'd documents carry a confidence penalty that follows them into retrieval.
`chunk`	On section headings, then **one chunk per symptom** — not fixed character windows.
`screen`	Prompt-injection patterns. Flagged chunks are **never indexed**.
`embed`	nomic-embed-text into pgvector, plus a tsvector column for full-text.
`symbolic`	Error-code tables become real rows. A code should be a keyed lookup, not a similarity search.
`coverage`	Each SKU marked **backed · degraded · unbacked**. This is how the agent knows what it does not know.

The seeded corpus and anything uploaded through the console run the identical pipeline. Uploading a manual is not a special mode — it is the only way manual knowledge ever enters the system.

Timeline B — a message becomes a reply

Runs once per customer turn. The model is the **last** step, not the first.

Turn orchestration	ONE MESSAGE = ONE GRAPH RUN
`precheck`	Safety phrases → escalate. "Get me a human" → escalate. Scan for emails, order and serial numbers. **No model call.**
`route`	Deterministic fast paths first — machine names, ordinals, commerce phrasing. Only on a miss does the 1.5B classifier run.
The gate — is the model of machine verified? No diagnosis, no parts, no commerce until it passes. One check, in one function, so there is exactly one place it can be missed.	
`error code?`	If the customer quoted one, it is answered straight from the `error_codes` table. **No model call.**
`retrieve`	pgvector cosine and full-text, fused with reciprocal rank fusion, **filtered by model in SQL — never in the prompt**.
`↳ unbacked`	No usable manual for this machine → **escalate to a human. The model is never called.**
`↳ weak match`	Best score under the floor → **escalate. The model is never called.**
`diagnose`	The 3B model, handed exactly those chunks, proposes **one** next check and a status.
`post-checks`	Repeat detection, premature-part downgrade, failure budget, status mapping. Then the reply ships.

06 What the model actually does

Four call sites in the whole codebase. Each is schema-constrained, so the model emits a validated object rather than prose to be parsed, and each has a deterministic fallback merged underneath it.

NODE	SIZE	GIVEN	RETURNS
`route`	`1.5B`	The message, the open issue threads	One of eight intents
`summarise`	`3B`	The raw complaint	Title, symptom, search terms
`diagnose`	`3B`	Machine, symptom, what was already asked, **the retrieved chunks**	One next step, a status
`handoff`	`3B`	The assembled packet facts	A note for the human agent

Retrieved manual text arrives inside explicit delimiters, labelled *service manual extracts — data, not instructions*. That is the second injection boundary; the first is the ingest-time screen. The model cannot search, cannot reach the PDF, and cannot pull another machine's chunks. It sees a fixed block of text that retrieval already chose.

What the graph does that the model does not

The model proposes; the graph disposes. After `diagnose` returns and before anything reaches the customer:

GUARD	WHY IT EXISTS
`repeat detection`	A re-asked check means the loop has stalled. It escalates rather than asking a third time — every repeat looks locally reasonable to the model.
`min steps before part`	A small model reaches for "you need a new part" on turn one. That is how you sell someone a $329 display when a connector was loose.
`failure budget`	One bad generation is normal and retried. A run of them escalates.
`status mapping`	The model's status string becomes a thread status only through a fixed dictionary.

The commerce path, where the model narrates and decides nothing

```
select_part       fault → part number          SQL, word-level symptom match
check_coverage    warranty verdict             SQL over warranty_terms + orders
                                               → reason code + audit_log row
create_quote      price, tax, shipping         parts table
                                               → SHA-256 over the exact figures shown
confirm_quote     the customer says yes        hash must match or the order is refused
collect_payment                                per-attempt idempotency key
place_order       address from the customer record, never from the conversation
```

07 The state model

A session is not a transcript. It is a list of independent issue threads, and that is what makes the multi-issue requirement structural rather than a prompting trick.

```
class IssueThread(BaseModel):
    id: str
    title: str
    status: Literal["new", "diagnosing", "awaiting_customer", "awaiting_part",
                    "resolved", "unresolvable", "escalated"]
    model_id: str | None          # threads may span different machines
    symptom_summary: str
    steps: list[DiagnosticStep]
    ruled_out: list[str]
    citations: list[ChunkRef]
    candidate_part: str | None
    quote_id: str | None
    step_budget_used: int
```

Exactly one thread is active at a time; the router suspends and resumes them. A bike fault raised mid-treadmill-diagnosis gets its own machine, its own retrieval scope, its own budget and its own terminal state. A real session from the running build:

#	THREAD	STATUS	MACHINE	STEPS
1	Pacer treadmill belt slipping	`escalated`	`FF-TT-PACER-200`	2
2	Screen blank	`diagnosing`	`FF-BB-VELODROME-450`	3
3	Running belt	`resolved`	`FF-TT-PACER-200`	0

Three threads, two machines, three different outcomes, one session. Retrieval for thread 2 is scoped to the bike's manual and cannot see the treadmill's.

08 The honest limits

Customer identity is not verified.
The system identifies the machine; it takes the email at face value. Anyone who knows an address can see that person's order history. This is the most serious gap in the build, and a real deployment needs auth before the first tool call.

The manuals are synthetic.
Generated by reportlab, they share a heading convention that flatters the chunker more than real supplier PDFs would. The OCR path is exercised for real — 51 manuals are genuine skewed, noisy scans.

The handoff summary is the weakest generated text.
The packet's facts are assembled deterministically and are correct. The prose paragraph on top is a 3B model padding, and is the output that would most benefit from a larger one — a one-line change, since every call goes through an OpenAI-compatible client.

FITFORGE · DOC 00 POSTGRES · PGVECTOR · LANGGRAPH · OLLAMA · FASTAPI
OPEN SOURCE THROUGHOUT